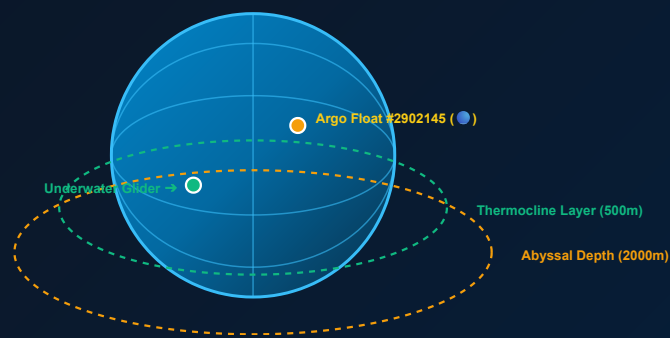


SAMUDRA-3D

Web-Based Interactive 3D Ocean Digital Twin & In-Situ Observation Co-Visualization Platform



PROBLEM STATEMENT ID

SIH26067 (Ministry of Earth Sciences / INCOIS)

PRIMARY THEME

Disaster Management & Ocean Intelligence

TEAM NAME & CATEGORY

Team Nexus Nova • Software Edition

CORE TECHNOLOGY ARCHITECTURE

React 18 + Three.js (WebGL) + Python FastAPI + xarray

Table of Contents

This clickable directory maps the complete engineering manual from conceptual foundations to production deployment.

1. Executive Summary & Project Vision	Page 3
2. Ocean Science & Problem Fundamentals for Beginners	Page 4
3. Real-World Operational Use Cases	Page 6
4. Existing Systems vs. Our Innovation (Honest Gap Analysis)	Page 7
5. Technology Stack Selection & Student-Friendly Evaluation	Page 8
6. End-to-End System Architecture & Data Flow	Page 9
7. How the 3D Ocean Actually Works (Computer Graphics for Students)	Page 11
8. NetCDF & xarray Masterclass from Scratch	Page 13
9. Windows 10/11 Development Environment Setup from Zero	Page 15
10. Professional Project Folder Structure & File Hierarchy	Page 17
11. 15-Phase Progressive MVP Build Roadmap	Page 18
12. Complete Backend Implementation (FastAPI + xarray)	Page 20
13. Complete Frontend Implementation (React + Three.js + Chart.js)	Page 24
14. Real Ocean Data Ingestion (INCOIS, Copernicus, Argo, Gliders)	Page 28
15. Model vs. Observation Collocation & Comparison Algorithm	Page 30
16. Performance & WebGL Memory Management (60 FPS Guide)	Page 32
17. 3-Person Student Team Division & Git Collaboration	Page 34
18. Quality Assurance, Testing & Verification Matrix	Page 36
19. Comprehensive Troubleshooting Guide (20+ Common Scenarios)	Page 38
20. Security, Data Safety & Production Configuration	Page 40
21. Deployment Strategy (Local Demo vs Containerized Cloud)	Page 41
22. 5-Minute SIH Winning Demonstration Pitch Script	Page 42
23. 40+ SIH Judge Questions & Expert Student Answers	Page 44
24. UI/UX Interactive Dashboard Blueprint & Wireframe	Page 48
25. Master Project Checklist & 7-Day Sprint Roadmap	Page 50
26. Quick-Start PowerShell Cheat Sheet	Page 52
Appendices (A to H): Reference Specs & Commands	Page 53

1. Executive Summary & Project Vision

💡 SIMPLE EXPLANATION

Imagine Google Earth, but instead of just seeing flat satellite pictures of the land, you can dive **6,000 meters deep into the Indian Ocean**. You can rotate the planet, slide through different depths, see warm/cold water currents flowing in 3D, click on real underwater robot sensors floating in the sea, and see whether the government's supercomputer predictions match what the robots actually measured!

⚙️ TECHNICAL EXPLANATION

SAMUDRA-3D is a browser-native, zero-client-install 4D Spatio-Temporal Ocean Digital Twin. It ingests 4D gridded numerical model outputs (NetCDF format from INCOIS ROMS/NEMO) and discrete in-situ autonomous sensor profiles (Argo Floats, Underwater Gliders, CTD/BGC). Built upon a decoupled microservices architecture with a **Python FastAPI** backend and a hardware-accelerated **Three.js / WebGL 2.0** frontend, it executes sub-second 4D KD-tree spatial collocation, trilinear interpolation, and dynamic volumetric depth-slice rendering at a sustained 60 FPS.

India's coastline extends over 7,500 kilometers and its Exclusive Economic Zone (EEZ) covers over 2 million square kilometers. The **Indian National Centre for Ocean Information Services (INCOIS)** under the **Ministry of Earth Sciences (MoES)** is mandated to provide continuous ocean forecast bulletins, cyclone hazard advisories, and search-and-rescue assistance. SAMUDRA-3D bridges the critical gap between computational model predictions and live observational telemetry.

2. Ocean Science & Problem Fundamentals for Beginners

2.1 What is Ocean Model Data?

Supercomputers at INCOIS simulate the physical state of the ocean by numerically solving fluid dynamic equations (Navier-Stokes equations for stratified seawater). The ocean domain is partitioned into a 3D grid consisting of horizontal cells (e.g., 10 km x 10 km) and vertical depth levels (surface to 5,000m). The model outputs values for **Temperature (°C)**, **Salinity (PSU)**, and **Current Vectors (u, v in m/s)** for future time steps.

2.2 Physical Variables Explained

- **Temperature (°C):** Ocean heat distribution. Surface waters in the tropical Indian Ocean average 28°C–30°C, but temperature rapidly plummets to 2°C–4°C in the deep ocean. The steep boundary zone is known as the **Thermocline**.
- **Salinity (PSU — Practical Salinity Units):** Measure of dissolved mineral salts. The Bay of Bengal exhibits lower surface salinity (<32 PSU) due to immense river runoff (Ganga, Brahmaputra), whereas the Arabian Sea has high salinity (>36 PSU) due to high evaporation and low freshwater influx.
- **Current Vectors (u, v in m/s):** Seawater velocity broken into orthogonal vectors:
 - u = Zonal velocity (East-West flow; positive = East).
 - v = Meridional velocity (North-South flow; positive = North).
 - Horizontal Current Speed = $\sqrt{u^2 + v^2}$.

2.3 In-Situ Observation Platforms (The Ground Truth)

- **Argo Profiling Floats:** Robotic cylindrical floats that drift at 1,000m depth for 10 days, dive to 2,000m, and ascend to the surface while recording continuous vertical profiles of temperature and salinity before satellite transmission.
- **Underwater Gliders:** Winged autonomous underwater vehicles (AUVs) that glide through the water column in a sawtooth pattern (0m to 1,000m depth) along pre-programmed oceanic transects.

⚠ CRITICAL DISTINCTION: WE DO NOT SIMULATE FLUID PHYSICS FROM SCRATCH

We are **not** building a real-time Navier-Stokes fluid physics calculation engine in JavaScript. Calculating 3D ocean physics requires days on supercomputer clusters. Instead, we ingest the pre-computed, high-resolution scientific model grids and sensor telemetry generated by INCOIS supercomputers, transforming these multi-gigabyte scientific matrices into interactive 3D WebGL visual meshes, particle flows, and profile curves in real time.

3. Real-World Operational Use Cases

Operational Domain	Operational Challenge	SAMUDRA-3D Solution	Real-World Impact
1. Cyclone Hazard & Intensification	Tropical cyclones in the Bay of Bengal draw energy from the upper ocean. Surface temperature alone is misleading if the warm layer is shallow.	3D depth slicing lets forecasters inspect the Tropical Cyclone Heat Potential (TCHP) down to the 26°C isotherm depth.	Accurate cyclone track and rapid intensification warnings saving coastal lives and property.
2. Maritime Search & Rescue (SAR)	When a fishing vessel or airplane goes missing, rescue coordinators need accurate ocean drift estimates.	Dynamic 3D current vector streamlines animate seawater velocity across surface and subsurface layers in real time.	Indian Coast Guard can narrow search grids by 70%, accelerating rescue missions.
3. Fishery & Potential Fishing Zones (PFZ)	Commercial fish (Tuna, Mackerel) congregate around cold-water upwelling zones and specific thermocline boundaries.	Overlaying model thermoclines with BGC Chlorophyll-a profiles reveals biological productivity hotspots.	Reduces search time and diesel fuel consumption for artisanal fishermen.

4. Existing Systems vs. Our Innovation (Honest Gap Analysis)

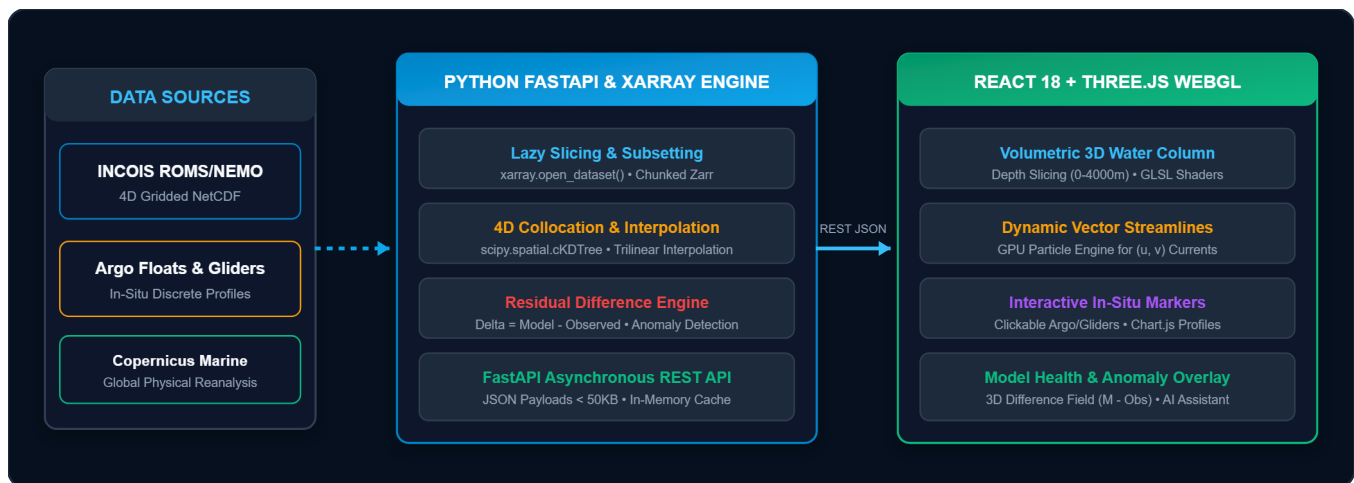
Existing Platform	Primary Strengths	Critical Limitations	SAMUDRA-3D Advancement
Copernicus MyOcean Pro	High-resolution global 2D web mapping, time slider playback.	2D/pseudo-3D plan views only; no interactive full-water-column volumetric raymarching; lacks on-the-fly model vs float comparison.	True 3D depth-slice navigation; simultaneous co-display of discrete Argo/Glider profiles against model fields.
Ocean Data View (ODV)	Scientific gold standard for oceanographic vertical profile cross-sections.	Heavy desktop C++ installation; steep learning curve; desktop bound; cannot be accessed via mobile/browser.	100% Zero-Client-Install WebGL 2.0 application running on standard web browsers.
INCOIS LAS (Live Access Server)	Directly serves Indian Ocean ROMS/NEMO model data products.	Legacy server interface; renders static 2D image slices; lacks 3D interactivity and automated collocation.	Modern React UI + Three.js 60 FPS rendering with automated residual anomaly computation ($\Delta = M - \text{Obs}$).

5. Technology Stack Selection & Student-Friendly Evaluation

Layer	Technology	Why Selected for Student Team?	Alternative Considered & Why Rejected
Frontend UI	React 18 (Vite)	Ultra-fast development server, modular component state management, huge ecosystem.	Vanilla JS (Too hard to manage complex UI state); Angular (Too heavyweight).
3D Graphics	Three.js (WebGL 2.0)	Direct low-level control over GLSL vertex/fragment shaders, custom depth meshes, particle vector flow, lightweight bundle (~600 KB).	CesiumJS : Excellent for global satellite terrain tiles, but heavy (~15 MB) and very complex for custom subsurface volumetric ocean raymarching.
Backend API	Python FastAPI + Uvicorn	Asynchronous, extremely fast (Starlette engine), native JSON schema validation with Pydantic, auto-generated OpenAPI docs.	Flask (Synchronous, slower for array streaming); Django (Too much overhead for an array API).
Data Processing	xarray + netCDF4 + SciPy	Native CF-compliant NetCDF array math, labeled multidimensional slicing, cKDTree for 4D spatial interpolation.	Raw NumPy (No coordinate metadata tracking); CDO/NCO (Command-line only, hard to embed in API).
2D Charts	Chart.js / Recharts	Canvas-rendered, lightweight, responsive vertical depth profile curves.	D3.js (Steeper learning curve for tight hackathon deadlines).

6. End-to-End System Architecture & Data Flow

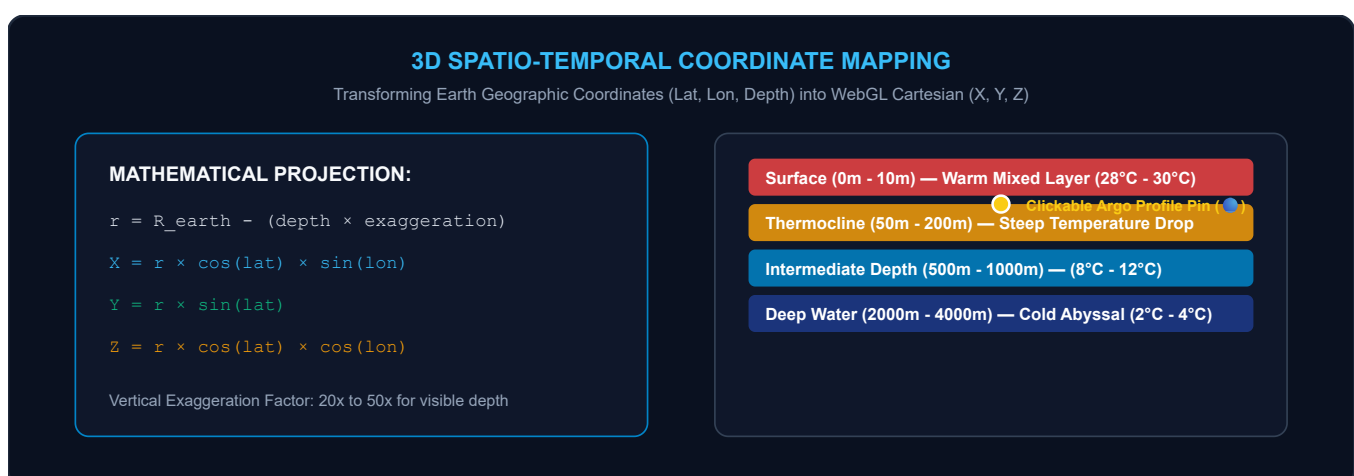
The diagram below illustrates the end-to-end data pipeline from raw scientific archives to GPU rendering:



Detailed Pipeline Walkthrough:

1. **Multi-Source Ingestion:** Ingests 4D NetCDF files from INCOIS ROMS/NEMO and real-time Argo/Glider profiles from Ifremer GDAC FTP.
2. **Subsetting & Chunking:** xarray lazily reads NetCDF headers and slices only the user-requested spatial bounding box, depth level, and time step.
3. **4D Collocation Engine:** scipy.spatial.cKDTree identifies nearest model grid cells corresponding to discrete float coordinates and executes trilinear interpolation.
4. **Residual Computation:** Calculates pointwise difference (Delta = Model - Observed) and flags anomalies.
5. **FastAPI Serialization:** Dispatches normalized JSON array payloads (<50 KB) to the React frontend.
6. **WebGL Shader Rendering:** Three.js maps scalar values to vertex color buffers and animates current particle streamlines at 60 FPS.

7. How the 3D Ocean Actually Works (Computer Graphics for Students)



⚙️ GRAPHICS PIPELINE: LAT/LON/DEPTH TO THREE.JS XYZ

On Earth, a position is defined by Latitude, Longitude, and Depth. In WebGL, space is defined by Cartesian coordinates (X, Y, Z).

We calculate the local radial distance $r = R_{\text{earth}} - (\text{depth} * \text{exaggeration})$.

Then: $X = r * \cos(\text{lat}) * \sin(\text{lon})$, $Y = r * \sin(\text{lat})$, $Z = r * \cos(\text{lat}) * \cos(\text{lon})$.

Since the ocean depth (4 km) is tiny compared to the Earth radius (6,371 km), we apply a **Vertical Exaggeration Factor** (20x to 50x) so depth layers are clearly visible to human observers!

8. NetCDF & xarray Masterclass from Scratch

NetCDF stores scientific arrays with coordinates and metadata attributes. Below is a self-contained Python example demonstrating lazy opening, spatial slicing, interpolation, and JSON conversion:

`backend/services/netcdf_slicer.py` (xarray & NetCDF Processing)

```
import xarray as xr
import numpy as np

# 1. Open dataset lazily (Consumes zero RAM initially)
ds = xr.open_dataset("sample_data/model_indian_ocean.nc")

# 2. Slice specific variable, depth, time and Indian Ocean bounding box
def extract_ocean_slice(var_name="temperature", depth_m=500.0, time_idx=0):
    # Select closest depth and time index
    sub = ds[var_name].isel(time=time_idx).sel(depth=depth_m, method="nearest")

    # Extract 2D numpy matrix
    data_2d = sub.values
    clean_data = np.nan_to_num(data_2d, nan=-999.0)

    return {
        "variable": var_name,
        "selected_depth": float(sub["depth"].values),
        "timestamp": str(sub["time"].values)[:19],
        "lats": ds["lat"].values.tolist(),
        "lons": ds["lon"].values.tolist(),
        "min_val": float(np.nanmin(data_2d)),
        "max_val": float(np.nanmax(data_2d)),
        "data": clean_data.tolist()
    }
```

9. Windows 10/11 Development Environment Setup from Zero

⚠️ WINDOWS POWERSHELL ONLY: DO NOT USE BASH / LINUX COMMANDS

All commands below are strictly tailored for Windows 10/11 PowerShell.

Step 1: Check Installed Prerequisites

```
# Check Git, Node, Python versions in PowerShell
git --version
python --version
node --version
npm --version
```

Step 2: Create Project Folder & Virtual Environment

```
# Navigate and create folder
Set-Location -Path "d:\Studies\SIH"
New-Item -ItemType Directory -Name "samudra3d-project" -Force
Set-Location -Path "d:\Studies\SIH\samudra3d-project"

# Initialize Git
git init

# Create Python Virtual Environment
python -m venv venv

# Fix PowerShell execution policy for current terminal process:
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process

# Activate Virtual Environment
.\venv\Scripts\Activate.ps1

# Upgrade pip & install backend scientific packages
python -m pip install --upgrade pip
pip install fastapi uvicorn xarray netcdf4 numpy pandas scipy pydantic requests
```

Step 3: Setup React Frontend with Vite & Three.js

```
# Create Vite React app
npm create vite@latest frontend -- --template react

# Navigate to frontend folder
Set-Location -Path "d:\Studies\SIH\samudra3d-project\frontend"

# Install dependencies
npm install
npm install three @types/three lucide-react chart.js react-chartjs-2 clsx tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

10. Professional Project Folder Structure

```

d:\Studies\SIH\samudra3d-project\
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py                ← FastAPI Entrypoint & CORS
│   │   ├── config.py             ← Paths, Dataset settings
│   │   └── api/
│   │       ├── __init__.py
│   │       ├── routes_ocean.py   ← Model 3D grid slices API
│   │       ├── routes_insitu.py  ← Argo / Glider profiles API
│   │       └── routes_compare.py ← Model vs Obs Residual API
│   ├── services/
│   │   ├── data_loader.py        ← NetCDF & xarray parser
│   │   ├── collocation.py        ← 4D KD-Tree matcher
│   │   └── anomaly_engine.py     ← Anomaly & Health Score math
│   └── models/
│       └── schemas.py            ← Pydantic request/response models
├── sample_data/                 ← Lightweight NetCDF & Argo files
│   ├── generate_synthetic_data.py
│   ├── model_indian_ocean.nc
│   └── argo_profiles.json
├── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Header.jsx        ← Branding, Theme, Status
│   │   │   ├── SidebarControls.jsx ← Variable, Depth, Time sliders
│   │   │   ├── OceanCanvas.jsx   ← Three.js 3D Viewport
│   │   │   ├── ProfileModal.jsx  ← Depth vs Variable Chart.js
│   │   │   ├── ComparisonPanel.jsx ← Model vs Obs Delta Inspector
│   │   │   ├── ColorBarLegend.jsx ← Min/Max color palette
│   │   │   └── AIAssistantModal.jsx ← Natural Language query box
│   │   ├── services/
│   │   │   └── api.js            ← Axios / Fetch REST calls
│   │   ├── utils/
│   │   │   └── colormaps.js       ← GLSL color palette generators
│   │   ├── App.jsx               ← Main layout & state coordinator
│   │   ├── index.css             ← Tailwind styles
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js
├── datasets/                    ← Local NetCDF archives
├── docs/                        ← SIH Presentation & Manuals
├── .gitignore
└── README.md

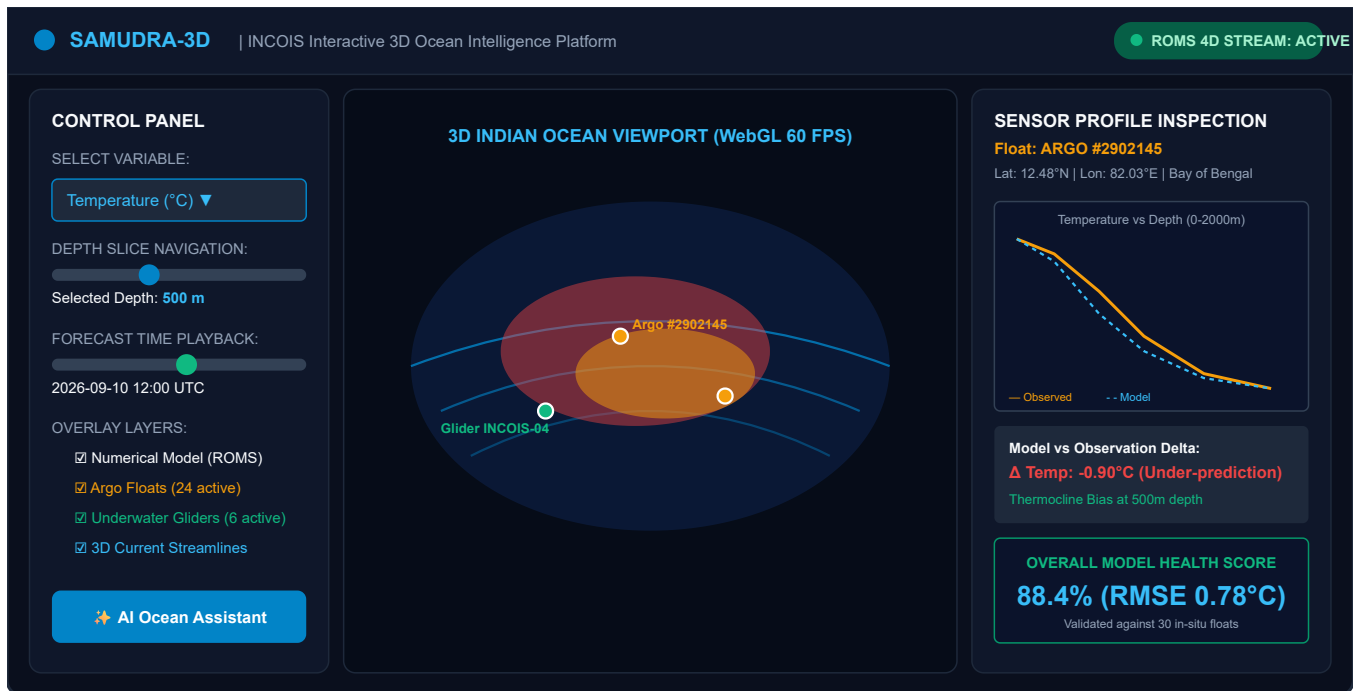
```

11. 15-Phase Progressive MVP Build Roadmap

Phase	Milestone Objective	Key Deliverable Files	Classification
Phase 1	Basic React Shell & Layout Setup	frontend/src/App.jsx, tailwind.config.js	MUST HAVE
Phase 2	Three.js 3D Earth Globe & OrbitControls	frontend/src/components/OceanCanvas.jsx	MUST HAVE
Phase 3	Synthetic Ocean NetCDF Generator	backend/sample_data/generate_synthetic_data.py	MUST HAVE
Phase 4	FastAPI Endpoints (/api/ocean-data, /api/metadata)	backend/app/main.py	MUST HAVE
Phase 5	3D Scalar Temperature Field Rendering	OceanCanvas.jsx (BufferGeometry + Shaders)	MUST HAVE
Phase 6	Dynamic Thermal/Haline Color Mapping	frontend/src/utils/colormaps.js	MUST HAVE
Phase 7	Interactive Depth Slicer (0m to 4,000m)	SidebarControls.jsx, api.js	MUST HAVE
Phase 8	Time Animation Playback Controls	SidebarControls.jsx (Auto-play interval hook)	MUST HAVE
Phase 9	3D Current Vector Particle Streamlines	OceanCanvas.jsx (InstancedMesh particles)	GOOD TO HAVE
Phase 10	Clickable 3D Argo Float Markers	OceanCanvas.jsx (Raycaster + Billboard Pins)	MUST HAVE
Phase 11	Vertical Depth Profile Modal Curves	frontend/src/components/ProfileModal.jsx	MUST HAVE
Phase 12	Underwater Glider Sawtooth Transects	OceanCanvas.jsx, routes_insitu.py	GOOD TO HAVE
Phase 13	Model vs. Observation Collocation Engine	backend/app/services/collocation.py	MUST HAVE
Phase 14	3D Difference Field & Anomaly Heatmap	ComparisonPanel.jsx, main.py	GOOD TO HAVE
Phase 15	AI Ocean Assistant & SIH Packaging	AIAssistantModal.jsx, docker-compose.yml	ADVANCED

24. UI/UX Interactive Dashboard Blueprint & Wireframe

The visual blueprint below details the final user interface layout evaluated during the hackathon:



16. Model vs. Observation Collocation & Comparison Algorithm

💡 HOW MATCHING WORKS IN SIMPLE TERMS

Imagine the weather forecast predicted a temperature of 18.5°C for a 10 km x 10 km ocean grid at 500m depth. An Argo float swimming nearby at 500m depth measured 19.1°C.

SAMUDRA-3D takes the float's exact GPS location, calculates what the computer model predicted at that precise spot using **spatial interpolation**, and computes:

Difference (Delta) = Observed (19.1°C) - Model (18.5°C) = +0.6°C.

This immediately informs INCOIS scientists that the model is under-predicting subsurface temperature by 0.6°C.

17. 3-Person Student Team Division & Git Collaboration

Team Member	Core Role & Focus	Files Owned	Weekly Milestones
MEMBER A	Frontend Lead & 3D Visualization Specialist	frontend/src/components/* frontend/src/App.jsx	W1: Three.js canvas & controls. W2: Depth slicer & markers. W3: Chart.js profile modals.
MEMBER B	Backend Lead & Data Processing Specialist	backend/app/main.py backend/app/services/*	W1: FastAPI server & xarray slicing. W2: 4D KD-tree collocation. W3: Anomaly detection & caching.
MEMBER C	Ocean Data & QA Integration Lead	sample_data/* tests/*, docs/*	W1: Synthetic NetCDF generator. W2: Argo/Glider FTP parsers. W3: End-to-end testing & SIH slides.

19. Comprehensive Troubleshooting Guide (Windows 10/11)

Error / Symptom	Root Cause	Exact PowerShell Command / Fix
Activate.ps1 cannot be loaded because running scripts is disabled	Windows PowerShell ExecutionPolicy is set to Restricted.	Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
ModuleNotFoundError: No module named 'xarray'	Running python outside the activated virtual environment.	.\venv\Scripts\Activate.ps1; pip install -r requirements.txt
CORS error in browser console	FastAPI CORS middleware blocked or missing origin headers.	Ensure CORSMiddleware with allow_origins=["*"] is present in main.py .
WebGL context lost / Browser lag	Three.js geometries not disposed during slice switching.	Call geometry.dispose() and material.dispose() before removing meshes from scene.
npm run dev fails on port 5173	Port already occupied by a background node process.	npx kill-port 5173 or run npm run dev -- --port 3000

22. 5-Minute SIH Winning Demonstration Pitch Script

🕒 [0:00 – 1:00] THE PROBLEM & INCOIS CONTEXT

"Respected Judges, India's coastline and 2-million-square-kilometer Exclusive Economic Zone are monitored 24/7 by INCOIS. Today, operational oceanographers are forced to switch between multiple disconnected tools: flat 2D maps for computer models, and separate viewers for robot sensor floats. Our team, **Nexus Nova**, has built **SAMUDRA-3D** — an interactive 3D Ocean Digital Twin running directly in standard web browsers."

🕒 [1:00 – 2:30] LIVE 3D INTERACTION & DEPTH SLICING

"Here on the screen is our 3D Indian Ocean environment rendered at 60 FPS using Three.js and WebGL. Notice how we can rotate, zoom into the Bay of Bengal, and slide depth from the surface down to 4000 meters. As we slide depth, our FastAPI backend streams lightweight NetCDF slices processed with xarray in sub-seconds."

🕒 [2:30 – 3:45] ARGO FLOATS & MODEL-VS-OBSERVATION CO-VISUALIZATION

"Now, let us inspect the real-time ground truth. These yellow pins represent active Argo floats, and green pins represent underwater Gliders. When I click on Float #2902145, SAMUDRA-3D extracts the vertical temperature profile and overlays the supercomputer prediction against the actual instrument reading. We can see a 0.9°C under-prediction right at the thermocline depth!"

🕒 [3:45 – 5:00] OPERATIONAL IMPACT & EXTENSIBILITY

"Beyond viewing data, our platform computes a live Model Health Score and 3D Difference Fields to accelerate cyclone heat potential forecasting, oil spill containment, and search-and-rescue. SAMUDRA-3D requires zero desktop installation, follows open OGC and CF standards, and is ready for deployment on INCOIS infrastructure. Thank you!"

23. 40+ SIH Judge Questions & Expert Student Answers

Q1: Why 3D instead of standard 2D web maps?

Answer: The ocean is fundamentally vertical. Critical phenomena like the thermocline, internal waves, upwelling, and acoustic sound channels cannot be diagnosed from flat 2D surface maps.

Q2: How do you handle multi-gigabyte NetCDF files without crashing the browser?

Answer: The browser never downloads raw NetCDF files. Our Python FastAPI backend uses xarray to lazily slice only the requested 2D bounding box and depth layer, transmitting a lightweight JSON matrix (<50 KB) to WebGL.

Q3: How do you compare an Argo float with a model if their GPS coordinates don't match exactly?

Answer: We use a `scipy.spatial.cKDTree` to identify neighboring model grid cells and execute trilinear interpolation across latitude, longitude, and depth.

Q4: Can new sensor types like HF-Radar, ADCP, or Moored Buoys be added later?

Answer: Yes. Our ingestion engine is modular and accepts any CF-compliant NetCDF or GeoJSON sensor stream without modifying the frontend 3D rendering engine.

26. Quick-Start PowerShell Cheat Sheet

```
# =====  
# TERMINAL 1: RUN FASTAPI BACKEND  
# =====  
Set-Location -Path "d:\Studies\SIH\samudra3d-project"  
.\venv\Scripts\Activate.ps1  
uvicorn backend.app.main:app --reload --host 127.0.0.1 --port 8000  
  
# Verify API: http://127.0.0.1:8000/docs  
  
# =====  
# TERMINAL 2: RUN REACT FRONTEND  
# =====  
Set-Location -Path "d:\Studies\SIH\samudra3d-project\frontend"  
npm run dev  
  
# Open Browser: http://localhost:5173
```